

DOCUMENTATION TECHNIQUE ET FONCTIONNELLE

Depot de Medicaments

Systeme multi-agents de gestion de stock avec prediction saisonniere
de penurie — version academique (PFE) et version production

Projet de fin d'etudes — Master 2 Systemes Intelligents
Ecole Nationale d'Informatique (ENI), Antananarivo

Document genere le 22 septembre 2026

Sommaire

1. Contexte et objectifs du projet
2. Perimetre fonctionnel — les 10 ecrans
3. Architecture technique (systeme multi-agents)
4. Pipeline SMA de bout en bout
5. Modele predictif — formule du score de risque
6. Modele de donnees (schema MySQL complet)
7. Processus metier detailles
8. Securite : authentication JWT et controle d'accès par role (RBAC)
9. Securite de base et fiabilite operationnelle
10. Deploiement (developpement, dev hybride, production)
11. Journal de construction : bugs reels trouves et corriges
12. Choix techniques documentes
13. Limites assumees et hors perimetre
14. Structure du depot de code et identifiants de demonstration

1. Contexte et objectifs du projet

Ce projet modelise un **depot de médicaments communautaire**, et non une pharmacie commerciale. Cette distinction n'est pas cosmétique : elle change le vocabulaire du code et impose deux processus métier qui n'existeraient pas dans une pharmacie classique.

Aspect	Pharmacie commerciale	Depot de médicaments (ce projet)
Qui gere	Pharmacien diplome	Gerant / agent communautaire
Catalogue	Large, marques et generiques	Liste restreinte de médicaments essentiels generiques
But	Commercial	Sante publique, prix coutant
Supervision	Ordre des pharmaciens	Autorite sanitaire de district (Superviseur)
Recyclage	Absent	Retrait / destruction encadre et trace
Conformite	Champ de statut	Processus métier a part entiere

Conséquence directe pour le code : on **distribue** (jamais "on vend"), un médicament est **archive** (jamais supprime), un retrait de stock est un **processus de conformité trace** (motif, justification, utilisateur responsable) et non une simple correction de chiffre.

Objectifs

- Surveiller en continu le stock de chaque médicament par rapport a un seuil de securite personnalise (en jours de couverture).
- Anticiper le risque de rupture avec un score combinant une regle de reapprovisionnement (ROP), la variabilite de la demande, et un **facteur saisonnier calendaire**.
- Proposer automatiquement une commande (quantite, fournisseur) quand le risque devient eleve, **sans jamais retirer au gerant la decision finale**.
- Tracer integrelement le cycle de vie d'un médicament : reception en lot, distribution, ajustement, retrait/destruction conforme.
- Demontrer, par une simulation integree, que la saisonnalite de la demande peut etre anticipee avant qu'elle n'apparaisse dans l'historique recent.

Principe central du projet : le systeme multi-agents (SMA) propose, il ne decide jamais seul. La validation finale (ajustement, annulation ou confirmation d'une commande) reste toujours un geste humain du gerant.

2. Perimetre fonctionnel — les 10 ecrans

#	Ecran	Actions	Roles
1	Tableau de bord	Indicateurs cles, alertes recentes, acces rapides (lecture seule, polling 5s)	Gerant + Superviseur
2	Medicaments	Creation / modification / archivage (pas de suppression), choix du profil saisonnier, categories VED/ABC	Gerant (CRUD) / Superviseur (lecture)
3	Fournisseurs	Fiches fournisseurs (fiabilite), conditions par medicament (delai, prix)	Gerant (CRUD) / Superviseur (lecture)
4	Lots & peremption	Reception, correction, tri par date de peremption, code couleur d'urgence	Gerant (create/read/update) / Superviseur (lecture)
5	Retrait / Destruction	Enregistrement motive (motif, quantite, justification, responsable) — write-once, jamais modifie	Gerant (create/read) / Superviseur (lecture)
6	Commandes	Creation manuelle, ajustement d'une proposition SMA, suivi livraison/retard/annulation	Gerant (create/read/update) / Superviseur (lecture)
7	Utilisateurs & roles	Comptes Gerant/Superviseur, reset de mot de passe, archivage	Superviseur uniquement (acces refuse au Gerant)
8	Mouvements	Simulateur ENTREE/DISTRIBUTION/AJUSTEMENT, declenche le pipeline SMA	Gerant (create) / Superviseur (lecture)
9	Alertes	Cartes trie'es par niveau de risque, action "traiter" avec confirmation nommee	Gerant (read + traiter) / Superviseur (lecture)
10	Previsions	Detail de chaque composante du score, courbe des 12 coefficients, mois courant en evidence	Gerant + Superviseur (lecture)

Principes d'ergonomie appliques a tous les ecrans

- Confirmation explicite **nommee** avant toute action sensible (jamais un "Etes-vous sur ?" generique).
- Retour visible (toast) apres chaque action, sur toutes les pages.
- Validation de formulaire en temps reel, au niveau du champ.
- Reversibilite par default : archivage plutot que suppression.
- Transparence : badge **IA** / **MANUELLE** visible sur chaque commande ; chaque composante du score de risque est affichee separement, jamais un chiffre opaque seul.

3. Architecture technique (systeme multi-agents)

```
frontend (React + Vite) ---> gateway (Node/Express) <---> Redis (pub/sub)
                        |
                        | REST (CRUD direct medicaments| ^ ^ ^
                        | mouvements, dashboard, alertes)| | |
                        v                               | |
MySQL <--- agent-monitoring (FastAPI, 8001) --+ |
MySQL <--- agent-prediction (FastAPI, 8002) ----+
MySQL <--- agent-decision (FastAPI, 8003) ----+
```

Les 4 composants du SMA

Gateway (Node.js / Express, port 4000)

Point d'entree API unique. Expose les routes REST CRUD (medicaments, mouvements, tableau de bord, alertes en acces direct base ; lots, retraits, fournisseurs, commandes, utilisateurs en proxy vers les agents qui portent leur logique metier). Ecoute le bus Redis et orchestre le pipeline en declenchant les agents via HTTP. Porte aussi l'authentification (JWT) et le controle d'accès par role (RBAC).

Agent Monitoring (FastAPI, port 8001)

Surveille en continu le stock de chaque medicament actif. Compare les jours de stock restant au seuil de securite ; si en dessous, publie l'evenement Redis `stock:seuil_critique`. Porte aussi la logique metier des modules **Lots** (reception, correction, code couleur d'urgence de peremption) et **Retrait / Destruction** (creation d'un retrait de conformite qui genere toujours un mouvement RETRAIT lie).

Agent Prediction (FastAPI, port 8002)

Expose l'endpoint `POST /predire/{id}`, appelle par le gateway en reaction a l'evenement `stock:seuil_critique`. Calcule le score de risque (detail section 5), insere une ligne `PREDICTION`, puis publie `prediction:calculee` sur Redis.

Agent Decision (FastAPI, port 8003)

Expose l'endpoint `POST /traiter-prediction`, appelle par le gateway quand le niveau de risque est `ELEVE` ou `CRITIQUE`. Choisit le fournisseur le plus fiable pour ce medicament, calcule une quantite proposee, cree une `COMMANDE` (source `IA`, statut `EN_ATTENTE`) et une `ALERTE` (statut `OUVERTE`). Porte aussi la logique des modules **Fournisseurs**, **Commandes** et **Utilisateurs** (y compris le hachage bcrypt des mots de passe).

Regle d'architecture stricte : les agents ne s'appellent JAMAIS directement entre eux, ni en HTTP ni en Redis. Seul le **gateway** ecoute le bus Redis et decide de declencher l'etape suivante via un appel HTTP a l'agent concerne. Chaque agent se contente de publier son resultat sur Redis a la fin de son traitement, sans jamais s'y abonner lui-meme. Ce decouplage permet de remplacer ou faire evoluer un agent (par exemple, le modele de prediction) sans toucher au code des autres. Chaque agent possede sa propre connexion MySQL independante.

Choix techniques de la stack

Composant	Choix
Frontend	React + Vite, organise par module metier (<code>frontend/src/modules/<nom>/</code> = page + tableau + formulaire)
Gateway	Node.js / Express
Agents	3 x FastAPI (Python), chacun avec sa propre connexion base de donnees
Base de donnees	MySQL 8, schema relationnel, requetes preparees (pas d'ORM lourd)
Bus d'evenements	Redis (pub/sub)
Deploiement	Docker Compose (dev) + <code>docker-compose.prod.yml</code> dedie (production)

4. Pipeline SMA de bout en bout

1. Un mouvement est enregistré via `POST /api/mouvements` (gateway) : écriture du mouvement, recalcul de `stock_actuel` , puis appel HTTP à l'agent Monitoring (`POST /verifier-stock`).
2. L'agent Monitoring compare les jours de stock restant (`stock_actuel / consommation_moyenne_journaliere`) au seuil `stock_securite_jours` . Si en dessous, il publie `stock:seuil_critique` sur Redis.
3. Le gateway (abonne à Redis) reçoit l'événement et appelle l'agent Prediction (`POST /predire/{id}`).
4. L'agent Prediction calcule le score de risque (formule section 5), insère une ligne `PREDICTION` , publie `prediction:calculee` .
5. Le gateway reçoit l'événement ; si `niveau_risque` est `ELEVE` ou `CRITIQUE` , il appelle l'agent Decision (`POST /traiter-prediction`).
6. L'agent Decision crée la `COMMANDE` (source `IA` , statut `EN_ATTENTE`) et l' `ALERTE` (`OUVERTE`).
7. Le frontend fait du polling toutes les 5 secondes sur `GET /api/alertes` (Tableau de bord + écran Alertes) pour un affichage quasi temps réel.
8. Le gerant **ajuste** la commande proposée (quantité, statut, fournisseur) depuis l'écran Commandes : cet ajustement est la validation, il n'y a pas de bouton "valider" séparé. Passer le statut à `LIVREE` crée automatiquement un mouvement `ENTREE` et augmente `stock_actuel` .

Diagramme de sequence (acteurs)

Trois acteurs interviennent : le **Gerant du depot** (usage quotidien), le **Superviseur de district** (contrôle/audit, comptes et prévisions), et le **SMA** lui-même comme acteur automatisé qui déclenche des cas d'usage sans intervention humaine. La dernière étape du pipeline (l'ajustement de la commande) est toujours une action humaine — le SMA propose, il ne décide jamais seul à la place du gerant.

5. Modele predictif — formule du score de risque

Un modele de point de commande (ROP) classique compare le stock restant a un seuil de securite, a partir de la consommation recente. Sa limite reelle : il ignore totalement le calendrier — un antipaludeen et un antitussif seraient traites de la meme facon en janvier comme en juillet, alors que leur demande reelle suit des cycles opposes selon la saison. Ce projet introduit un **facteur saisonnier calendaire** pour corriger cela.

Profils saisonniers

Chaque medicament est rattache a un profil parmi quatre, chacun portant un coefficient par mois (1 a 12) stocke dans une table dediee — ajouter un nouveau profil ne necessite aucune migration de schema :

- `STABLE` — pas d'effet calendaire connu (coefficient 1.00 toute l'annee)
- `RHUME_SAISON_FRAICHE` — pic mai-septembre
- `PALUDISME_SAISON_PLUIES` — pic novembre-avril
- `DIARRHEE_SAISON_CHAUDE` — pic octobre-janvier

Important : les coefficients actuels sont illustratifs et plausibles, pas calcules a partir de vraies statistiques epidemiologiques. Calibrer ces coefficients avec de vraies donnees est une evolution prevue, pas un prerequis de cette version.

Formule de calcul (chaque etape reste affichable separement)

```
1. Score ROP (point de commande)
marge_jours = (stock_actuel / consommation_moyenne_journaliere) - stock_securite_jours
score_rop = clamp(0,1, (horizon - marge_jours) / horizon) # horizon = 14 jours par default

2. Score ML (proxy statistique)
variabilite_demande = ecart_type_consommation / consommation_moyenne
score_ml = (score_rop * 0.7) + (variabilite_demande * 0.3)

3. Score ML ajuste (facteur saisonnier)
score_ml_ajuste = min(1, score_ml * facteur_saisonnier[mois_courant])

4. Score final (ponderation VED)
poids_ved = { V: 1.3 (Vital), E: 1.1 (Essentiel), D: 0.9 (Desirable) }
score_final = min(1, score_ml_ajuste * poids_ved[categorie_ved])
```

`consommation_moyenne_journaliere` et `ecart_type_consommation` sont calcules a partir des mouvements `DISTRIBUTION` des 30 derniers jours pour ce medicament. Seuils de `niveau_risque` appliques sur `score_final` (configurables via variables d'environnement) :

score_final	niveau_risque
< 0.40	FAIBLE
< 0.60	MOYEN
< 0.85	ELEVE
≥ 0.85	CRITIQUE

Exemple chiffre — Paracetamol sirop pediatrique

Stock actuel 40 unites **Stock de securite** 5 jours **Consommation/jour** 6 unites, ecart-type 3 **Profil** RHUME, categorie Essentiel

```
jours_stock_restant = 40 / 6 ~= 6,7 jours
score_rop = (14 - (6,7 - 5)) / 14 ~= 0,88
variabilite_demande = 3 / 6 = 0,5
score_ml = 0,88 x 0,7 + 0,5 x 0,3 ~= 0,77
facteur_saisonnier (juin, profil RHUME) = 1,5
```

```
score_ml_ajuste      = min(1 ; 0,77 x 1,5) = 1,0  
score_final (poids Essentiel = 1,1) = min(1 ; 1,0 x 1,1) = 1,0  -> CRITIQUE
```

En juin, un antitussif au profil RHUME_SAISON_FRAICHE voit son score multiplié par 1,5 même si son stock est objectivement stable — le système anticipe la hausse saisonnière avant qu'elle n'apparaisse dans les mouvements récents.

Point d'extension machine learning (non implémenté)

La fonction de calcul du score ML est isolée dans `agents/prediction/scoring.py`, avec un commentaire explicite indiquant qu'un futur modèle **XGBoost** (Gradient Boosting sur arbres de décision) pourrait la remplacer — entraîne sur l'historique réel pour apprendre les poids plutôt que de les fixer à la main (0.7/0.3 actuellement), avec repli automatique sur le calcul heuristique actuel si l'historique d'un médicament est insuffisant. **Ce modèle n'est PAS implémenté dans cette version** : c'est une évolution discutée avec l'encadreur, pas une livraison de ce projet.

6. Modele de donnees (schema MySQL complet)

13 tables au total (12 tables metier du cahier des charges initial + 1 table ajoutee pour la production : `REFRESH_TOKEN`).

Vue d'ensemble

Table	Role
MEDICAMENT	Fiche du medicament essentiel : profil saisonnier, categories VED/ABC, stock, seuil de securite. Archive plutot que supprimer.
PROFIL_SAISONNIER / COEFFICIENT_SAISONNIER	Referentiel des motifs calendaires et de leurs 12 coefficients mensuels — table normalisee, extensible sans migration.
FOURNISSEUR / MEDICAMENT_FOURNISSEUR	Partenaires d'approvisionnement et conditions (delai, prix) propres a chaque couple medicament-fournisseur.
LOT	Reception physique : numero, date de peremption, quantite.
MOUVEMENT	Historique complet des variations de stock (ENTREE, DISTRIBUTION, AJUSTEMENT, RETRAIT), horodate. Quantite toujours positive.
RETRAIT_DESTRUCTION	Processus de conformite : motif, quantite, justification, utilisateur responsable, rattache a un lot et a un mouvement RETRAIT.
COMMANDE	Commande fournisseur, manuelle ou proposee par le SMA, avec son statut.
PREDICTION / ALERTE	Resultat du calcul de risque (chaque composante) et notification associee.
UTILISATEUR	Comptes Gerant / Superviseur, mot de passe hache en bcrypt.
REFRESH_TOKEN	Ajoutee en phase production : refresh tokens haches, permet la revocation reelle des sessions.

Modele logique de donnees (notation Merise)

```
PROFIL_SAISONNIER (id_profil, nom_profil, description)
COEFFICIENT_SAISONNIER (#id_profil, mois, coefficient)
FOURNISSEUR (id_fournisseur, nom_fournisseur, fiabilite_score)
MEDICAMENT (id_medicament, nom_medicament, dci, code_essentiel, forme_pharmaceutique,
  categorie_ved, categorie_abc, #id_profil_saisonnier, stock_actuel,
  stock_securite_jours, prix_unitaire, actif)
MEDICAMENT_FOURNISSEUR (#id_medicament, #id_fournisseur, delai_livraison_moyen_jours,
  delai_livraison_max_jours, prix_achat)
LOT (id_lot, #id_medicament, #id_fournisseur, numero_lot, date_peremption, quantite_lot)
UTILISATEUR (id_utilisateur, nom_utilisateur, email, mot_de_passe_hash, role, actif)
MOUVEMENT (id_mouvement, #id_medicament, #id_lot, type_mouvement, quantite_mouvement,
  date_mouvement, source_mouvement)
RETRAIT_DESTRUCTION (id_retrait, #id_lot, #id_utilisateur, #id_mouvement, motif,
  quantite_retee, justification, date_retrait)
COMMANDE (id_commande, #id_medicament, #id_fournisseur, quantite_commandee, date_commande,
  date_livraison_prevue, date_livraison_reelle, statut_commande, source_commande)
PREDICTION (id_prediction, #id_medicament, date_prediction, horizon_jours, score_rop,
  score_ml, facteur_saisonnier, score_final, niveau_risque, modele_version)
ALERTE (id_alerte, #id_medicament, #id_prediction, niveau_risque, statut_alerte, date_creation)
REFRESH_TOKEN (id_refresh_token, #id_utilisateur, token_hash, date_creation,
  date_expiration, revoke)
```

Dictionnaire de donnees — champs cles

MEDICAMENT

Attribut	Type	Description
----------	------	-------------

categorie_ved	ENUM(V,E,D)	Criticite clinique : Vital / Essentiel / Desirable — ponderation 1.3 / 1.1 / 0.9 dans le score
categorie_abc	ENUM(A,B,C)	Classification par valeur/volume
stock_actuel	INT	Quantite en stock, recalculee a partir des mouvements
stock_securite_jours	DECIMAL	Nombre de jours de couverture souhaitees
actif	BOOLEAN	Faux si le medicament est archive

MOUVEMENT

Attribut	Type	Description
type_mouvement	ENUM	ENTREE / DISTRIBUTION / AJUSTEMENT / RETRAIT
quantite_mouvement	INT	Toujours positive (sauf AJUSTEMENT, delta libre) ; le sens depend de type_mouvement
source_mouvement	VARCHAR	Origine : manuel, reception_lot, commande_livree, etc.

PREDICTION / ALERTE

Attribut	Type	Description
score_rop / score_ml / score_final	DECIMAL(4,3)	Chaque composante du calcul, stockee separement pour transparence
facteur_saisonnier	DECIMAL(4,2)	Coefficient du mois courant selon le profil
niveau_risque	ENUM	FAIBLE / MOYEN / ELEVE / CRITIQUE
statut_alerte	ENUM	OUVERTE / TRAITEE (jamais fermee automatiquement)

COMMANDE

Attribut	Type	Description
statut_commande	ENUM	EN_ATTENTE / LIVREE / EN_RETARD / ANNULEE
source_commande	ENUM	IA (proposee par le SMA) / MANUELLE

REFRESH_TOKEN (ajoutee en phase production)

Attribut	Type	Description
token_hash	VARCHAR(255)	Hash du refresh token (jamais stocke en clair)
revoque	BOOLEAN	Permet la revocation immediate au logout ou a la rotation

7. Processus metier details

7.1 — Reception d'un lot

Le gerant enregistre un lot recu d'un fournisseur (numero de lot, date de peremption, quantite). Cette action cree automatiquement un mouvement **ENTREE** et augmente le stock du medicament concerne — une seule saisie plutot que deux etapes separees.

7.2 — Distribution

Chaque distribution a un patient ou une structure de sante relais diminue le stock. C'est cette consommation qui alimente le calcul du risque de rupture.

7.3 — Retrait / Destruction (conformite)

Un medicament perime, non utilise ou juge non conforme n'est jamais simplement efface du stock. Le gerant enregistre un retrait motive, rattache au lot concerne et a la personne qui le valide : le systeme cree un mouvement **RETRAIT** et une fiche **retrait_destruction** tracable — exigence reelle de conformite pour un depot de medicaments.

7.4 — Commande fournisseur

Une commande peut etre proposee automatiquement par l'Agent Decision (score de risque eleve) ou creee manuellement par le gerant. Une proposition automatique reste modifiable avant d'etre lailsee suivre son cours — l'ajustement est l'action de validation, il n'y a pas de bouton "valider" separe. Une commande livree declenche automatiquement l'entree en stock correspondante.

7.5 — Traitement d'une alerte

Une alerte de risque eleve est confirmee comme traitee par le gerant une fois la situation verifiee ou la commande passee — jamais fermee automatiquement par le systeme.

8. Securite : authentication JWT et controle d'accès par rôle (RBAC)

8.1 — Authentication JWT

Ajoutée lors du passage en conditions de production réelle (la version académique n'implémentait volontairement pas d'authentification, conforme au périmètre du cahier des charges initial).

- **Hachage des mots de passe** : bcrypt (remplace le SHA-256 démonstratif de la version académique), gère exclusivement par l'agent Decision (seul module à manipuler la table UTILISATEUR).
- **POST /api/auth/login** : email + mot de passe -> vérification bcrypt (appel interne gateway -> agent Decision) -> émission d'un access token JWT courte durée (15 min par défaut) et d'un refresh token longue durée (7 jours), stocke en cookie httpOnly et hache en base (table `REFRESH_TOKEN`) pour permettre la révocation.
- **POST /api/auth/refresh** : rotation du refresh token à chaque appel, vérifie la non-révocation.
- **POST /api/auth/logout** : révoque le refresh token courant.
- **Frontend** : écran de connexion, contexte d'authentification React, déconnexion automatique à l'expiration (intercepteur sur les réponses 401 qui tente un refresh puis déconnecte si échec).
- Le secret de signature (`JWT_SECRET`) est exclusivement fourni par variable d'environnement, jamais en dur dans le code.

8.2 — Matrice RBAC exacte (appliquée route par route, côté serveur)

Ressource	Gerant	Superviseur
Tableau de bord	Lecture	Lecture
Medicaments	CRUD (sans delete, archivage ok)	Lecture
Fournisseurs	CRUD	Lecture
Lots & peremption	Create / Read / Update	Lecture
Retrait / Destruction	Create / Read	Lecture
Commandes	Create / Read / Update	Lecture
Utilisateurs & rôles	Aucun accès (403)	CRUD
Mouvements	Create (+ read)	Lecture
Alertes	Read + traiter	Lecture
Previsions	Lecture	Lecture

Principe cle : la vérification qui compte est celle du serveur (middleware `autoriser(...roles)` applique à chaque route du gateway, retourne 403 si le rôle du token ne correspond pas). Le frontend masque en plus les entrées de menu et actions non autorisées pour l'ergonomie, mais cela ne remplace jamais la vérification serveur.

Vérifie réellement pendant le développement : 403 confirme pour un Gerant sur `/api/utilisateurs`, 403 pour un Superviseur sur `POST /api/mouvements`, 401 sans jeton valide.

9. Securite de base et fiabilite operationnelle

9.1 — Securite de base

- **Secrets externalises** : mot de passe DB, `JWT_SECRET`, credentials Redis — tous en variables d'environnement via `.env` (jamais committe, deja dans `.gitignore`) avec un `.env.example` fourni.
- **CORS restreint** : origine configurable via `FRONTEND_ORIGIN` (plus de wildcard `*`) sur le gateway et les 3 agents FastAPI.
- **Validation des entrees** : schemas `zod` cote gateway sur les routes sensibles (ex. `POST /api/mouvements`, `POST/PUT /api/medicaments`), en plus des validations `Pydantic` deja presentes cote agents. Aucune concatenation SQL directe verifiee dans tout le code (uniquement des requetes preparees).
- **Limitation de debit** : `express-rate-limit` sur `POST /api/auth/login` (5 tentatives / 15 min par IP) pour limiter le bruteforce — teste reellement (code 429 obtenu apres 5 essais rapproches).

9.2 — Fiabilite operationnelle

- **Logs structures JSON** : `pino` cote gateway, module `logging` avec formatter JSON custom cote agents Python. Niveau configurable via `LOG_LEVEL`.
- **Healthchecks Docker Compose** sur tous les services (mysql, redis, gateway, 3 agents, frontend), avec redemarrage automatique en cas de crash (`restart: unless-stopped` en dev, `restart: always` en production).
- **Reconnexion automatique du bus Redis** cote gateway et agents (backoff sur coupure) — testee reellement en redemarrant le conteneur Redis pendant que le stack tournait : aucun crash, reconnexion automatique, pipeline refunctional immediatement apres. Un echec de publication est journalise en warning, jamais silencieux.
- **Sauvegarde MySQL quotidienne** : script de dump compresse avec rotation (variable `BACKUP_RETENTION_DAYS`), branche comme service Docker Compose dedie en production.

10. Deploiement

10.1 — Developpement (Docker Compose complet)

```
cd smart-medicament
docker compose up --build

# Peupler la base (une fois les conteneurs demarres)
cd db/seed
npm install
npm run seed:all # DB_HOST=localhost DB_PORT=3306 ...
```

Ouvrir <http://localhost:5173> — redirection automatique vers [/login](#).

10.2 — Mode dev hybride (hot-reload)

Infrastructure (mysql + redis) en conteneurs, gateway et frontend en local avec rechargement a chaud. Utile pour developper rapidement sans reconstruire d'image a chaque modification. Voir le README pour la sequence complete.

10.3 — Production (`docker-compose.prod.yml`)

Aspect	Dev	Production
Code source	Bind-mount pour hot-reload	Build-only, images figees
MySQL / Redis	Ports exposes a l'hote	Non exposes, uniquement joignables entre conteneurs
Politique de redemarrage	unless-stopped	always
Secrets	Valeurs par default dev	<code>.env</code> obligatoire, jamais commite
Sauvegarde	Non branchee	Service <code>backup</code> automatique (24h, rotation N jours)

```
cp .env.example .env
# editer .env avec de vraies valeurs (JWT_SECRET fort, mots de passe DB robustes)
docker compose -f docker-compose.prod.yml up -d --build
```

Variables d'environnement a configurer en production

Variable	Role
MYSQL_ROOT_PASSWORD	Mot de passe root MySQL
FRONTEND_ORIGIN	Origine exacte du frontend (CORS)
JWT_SECRET	Secret de signature JWT (long, aleatoire)
JWT_ACCESS_EXPIRES_IN	Duree de vie de l'access token (default 15m)
JWT_REFRESH_EXPIRES_IN_DAYS	Duree de vie du refresh token (default 7 jours)
LOG_LEVEL	Niveau de log
BACKUP_RETENTION_DAYS	Jours de dumps MySQL conservees (default 7)

Sauvegarde et restauration

```
# Sauvegarde manuelle (le service backup le fait deja automatiquement chaque jour)
docker compose -f docker-compose.prod.yml exec -T backup /app/backup.sh

# Restauration (attention : ecrase les donnees actuelles)
```

```
gunzip -c dump.sql.gz | docker compose -f docker-compose.prod.yml exec -T mysql \
mysql -uroot -p"$MYSQL_ROOT_PASSWORD" smart_medicament
```

11. Journal de construction : bugs reels trouves et corriges

Le pipeline n'a pas ete valide uniquement par relecture de code : il a ete reellement construit avec Docker (`docker compose build / up`), la base a ete peulee, et le pipeline a ete declenche par de vrais appels HTTP. Cette validation a mis en evidence deux bugs reels, corriges avant integration.

Bug 1 — Convention de signe incoherente sur `quantite_mouvement`

Le dictionnaire de donnees precise que `quantite_mouvement` doit toujours etre **positive** ; le sens (augmente ou diminue le stock) depend uniquement de `type_mouvement` . Le seed respectait cette regle, mais la route `POST /api/mouvements` du gateway et l'endpoint de retrait de l'agent Monitoring stockaient les mouvements `DISTRIBUTION / RETRAIT` avec une valeur **negative**.

Consequence observee : `SUM(quantite_mouvement)` (utilise par les 3 agents pour calculer la consommation moyenne journaliere) melangeait des valeurs positives (historique du seed) et negatives (nouveaux mouvements), produisant une moyenne **negative** qui desactivait silencieusement la detection de seuil critique (`sous_seuil_critique: false` alors que le stock etait objectivement bas).

Correction : les deux points d'ecriture stockent désormais toujours une quantite positive ; le sens est applique separement lors de la mise a jour de `stock_actuel` (delta signe calcule a part, jamais persiste tel quel, sauf pour `AJUSTEMENT` qui reste un delta libre).

Bug 2 — Double declenchement des agents Prediction et Decision

Le code initial faisait ecouter Redis **a la fois** par le gateway (orchestrateur) et par les agents Prediction/Decision eux-memes (chacun avait un thread d'arriere-plan abonne au canal correspondant). Resultat : un seul evenement `stock:seuil_critique` declenchait le calcul de score **deux fois** (une fois via l'auto-declenchement de l'agent, une fois via l'appel HTTP du gateway), et de meme pour `prediction:calculee` .

Consequence observee : un test reel (une distribution faisant passer un medicament sous son seuil) a produit **4 alertes et 4 commandes identiques** au lieu d'une seule.

Correction : les agents Prediction et Decision n'ecoutent plus Redis pour s'auto-declencher ; ils exposent uniquement leurs endpoints HTTP que le gateway appelle en reaction aux evenements qu'il observe. Les agents continuent de publier sur Redis a la fin de leur traitement (pour respecter le decouplage attendu), mais ne s'y abonnent plus. Reteste : un declenchement produit exactement une alerte et une commande.

Verification finale du stack complet

Apres les deux phases de construction, tous les services (`mysql` , `redis` , `agent-monitoring` , `agent-prediction` , `agent-decision` , `gateway` , `frontend`) sont `healthy` sans boucle de redemarrage. Le pipeline SMA a ete reteste de bout en bout avec authentification reelle :

```
POST /api/auth/login (GERANT)          -> jeton recu
POST /api/mouvements (Bearer <jeton>) -> DISTRIBUTION qui fait passer un
                                         medicament sous son seuil
                                         -> alerte CRITIQUE + commande IA
                                         EN_ATTENTE creees en quelques secondes
GET  /api/utilisateurs (role GERANT)    -> 403 (attendu, reserve au Superviseur)
```

12. Choix techniques documentes

Decision	Justification
Seed en Node.js (pas Python/SQL statique)	Reutilise <code>mysql2</code> , reste dans le meme ecosysteme que le gateway
~3800 mouvements historiques (vs ~28 500 évoques a titre indicatif)	Suffisant pour demontrer la saisonnalite, temps de generation maitrise
Frontend buildé statique + Nginx en production	Plus proche d'un deploiement reel qu'un serveur Node de dev
Verification du mot de passe centralisee dans l'agent Decision	Garde toute la logique metier Utilisateurs (et le hachage bcrypt) a un seul endroit ; le gateway ne manipule jamais de mot de passe en clair
Refresh token hache en base (jamais en clair)	Permet une revocation reelle (logout, rotation a chaque refresh) meme si la base est compromise
Sauvegarde MySQL en service Docker Compose dedie	Auto-suffisant dans l'ecosysteme Compose ; alternative cron documentee en commentaire pour un environnement qui le prefererait
Healthcheck frontend cible <code>127.0.0.1</code> et non <code>localhost</code>	Bug decouvert en test reel : l'image nginx:alpine resolvait <code>localhost</code> en IPv6 sans listener, faisant echouer le healthcheck malgre un service fonctionnel

13. Limites assumees et hors perimetre

- **Pas de calibration des coefficients saisonniers** avec de vraies donnees epidemiologiques — restent illustratifs, comme prevu par le cahier des charges. Deux pistes envisagees pour une evolution future : donnees ouvertes (bulletins OMS/Ministere de la Sante) ou apprentissage progressif a partir de l'historique reel de distribution.
- **Pas de modele XGBoost entraine** : le score reste un indicateur statistique fait main ; la fonction de calcul est isolee pour permettre un remplacement futur, avec repli automatique si l'historique est insuffisant.
- **Pas de support multi-depot** : un seul depot modelise pour cette version.
- **Pas de suite de tests automatises** (unitaires/integration) : la validation s'est faite par des tests manuels reels (rebuild Docker + appels HTTP) a chaque etape sensible — c'est cette methode qui a permis de trouver les deux bugs de la section 11, qu'une simple relecture de code aurait probablement laisses passer.
- **Rate limiting en memoire** par instance du gateway : suffisant pour un seul conteneur (situation actuelle), ne serait pas partage si le gateway etait execute en plusieurs instances (hors perimetre demande).

14. Structure du depot de code et identifiants de demonstration

Arborescence

smart-medicament/	
docker-compose.yml	stack de developpement
docker-compose.prod.yml	stack de production
.env.example	variables sensibles (exemple, jamais commite)
README.md	architecture, instructions de lancement
JOURNAL.md	journal de construction detaille
db/	
migrations/	
001_schema.sql	schema MySQL complet (12 tables)
002_auth.sql	table REFRESH_TOKEN (ajoutee en production)
seed/	
seed_base.js	profils, coefficients, fournisseurs, medicaments, lots, utilisateurs
seed_mouvements.js	genere l'historique de mouvements avec saisonnalite
backup/	
backup.sh	dump MySQL quotidien avec rotation
gateway/	Express - API unique, auth JWT, RBAC, orchestration Redis
src/	
routes/	10 fichiers de routes (1 par ressource)
middleware/	auth.js, rbac.js, validate.js
services/	logger.js, orchestrator.js, agentClient.js
agents/	
monitoring/	FastAPI (8001) - stock, lots, retraits
prediction/	FastAPI (8002) - scoring.py = coeur du calcul de risque
decision/	FastAPI (8003) - fournisseurs, commandes, utilisateurs, hachage bcrypt
frontend/	
src/modules/<10 ecrans>/	chaque module = page + tableau + formulaire
src/modules/auth/	ecran de connexion, contexte d'authentification

Identifiants de demonstration (apres seed)

Role	Email	Mot de passe
Gerant	gerant.depot@smart-medicament.example	depot1234
Superviseur	superviseur.district@smart-medicament.example	depot1234

Fin du document — Depot de Medicaments, systeme multi-agents de gestion de stock avec prediction saisonniere de penurie.